Computação Gráfica
Guia de Estudo para a Prova

Transformações · Câmera e Projeção (MVP) · Iluminação · Texturas

Feito a partir dos slides, do livro (cap. 2.2–2.6, 3, 5, 7 e 8.3), do código da disciplina e das provas anteriores

Como usar este guia

O guia supõe que você sabe programar mas perdeu boa parte das aulas. Ele começa do zero (o que é um pixel, o que a GPU faz) e chega ao nível das provas. Não há questões de escrever código: as provas pedem para **explicar imagens com defeito**, **dar coordenadas de textura** e **calcular matrizes**.

Se a questão pede…	Leia
Nomear vértices e dar coordenadas de textura de um cubo, rua, dado…	Parte 6.1 a 6.4
Explicar textura borrada, serrilhada, com "linha" de transição, distorcida	Parte 6.5 e 6.6, e a Parte 7
Explicar por que uma imagem iluminada ficou estranha (faces pretas, sem brilho)	Parte 5 e a Parte 7
Explicar degradê de cores num quadrado	Parte 1.5
Montar a matriz final (MVP), dar posições finais e dizer se o triângulo aparece	Partes 2, 3 e 4, e a Parte 8 (planilha)

> **Na véspera, se o tempo for curto**
> Leia a Parte 9 (o que caiu em cada prova), a Parte 7 (tabela de diagnóstico) e a Parte 10 (folha de fórmulas). Depois faça uma questão de MVP com a planilha (Parte 8).

Sumário

Parte 1 — Como o computador desenha 3D

1.1 A imagem é uma matriz de pixels
1.2 CPU e GPU
1.3 O que o programa manda para a GPU
1.4 O pipeline gráfico, passo a passo
1.5 Interpolação de atributos
1.6 Lendo código que aparece na prova

Parte 2 — A matemática necessária

2.1 Pontos e vetores
2.2 Produto escalar
2.3 Produto vetorial
2.4 Matrizes e multiplicação
2.5 Coordenadas homogêneas

Parte 3 — Transformações geométricas

3.1 Translação, escala e rotação
3.2 Inversas
3.3 Composição: a ordem importa
3.4 Girar ou escalar em torno de um ponto
3.5 Traduzindo o enunciado em matrizes

Parte 4 — Câmera, projeção e MVP

4.1 A viagem de um vértice
4.2 A matriz de visualização (câmera)
4.3 A matriz de projeção perspectiva
4.4 NDC e viewport (pixels)
4.5 O triângulo aparece? Recorte e oclusão
4.6 Mudando a câmera de propósito
4.7 Exemplo completo, feito à mão
4.8 Receita da questão de MVP

Parte 5 — Iluminação e sombreamento

5.1 Fontes de luz
5.2 O modelo de Phong
5.3 Normais
5.4 Sombreamento: constante, Gouraud, Phong
5.5 Sombras

Parte 6 — Texturas

6.1 Conceitos
6.2 Dando coordenadas de textura
6.3 Repetição e modos de wrap
6.4 Diagnosticando mapeamentos errados
6.5 Correção de perspectiva
6.6 Filtragem: nearest, bilinear, mipmap, anisotrópica
6.7 Bump mapping, environment mapping e transparência

Parte 7 — Tabela de diagnóstico de imagens

Parte 8 — A planilha, passo a passo

Parte 9 — As três provas, questão por questão

Parte 10 — Glossário e folha de fórmulas

Parte 1 — Como o computador desenha uma cena 3D

1.1 A imagem é uma matriz de pixels

A tela é uma grade de pontos chamados **pixels** (picture elements). Uma imagem assim é chamada de **raster** (matricial). A **resolução** é largura × altura: 800 × 600, 1024 × 768 etc. A razão largura/altura é a **proporção** (aspect ratio): 800/600 = 1,33.

A imagem que vai aparecer fica guardada na memória de vídeo, no **framebuffer**, que tem duas partes:

- **Buffer de cor:** a cor (R, G, B) de cada pixel. Com 24 bits por pixel (8 por canal) há cerca de 16 milhões de cores.
- **Buffer de profundidade (Z-buffer):** para cada pixel, a distância do que está desenhado ali. É ele que decide quem fica na frente quando dois objetos caem no mesmo pixel.

Para animação há dois buffers de cor (**double buffering**): a placa mostra um enquanto o próximo quadro é desenhado no outro, e depois eles trocam.

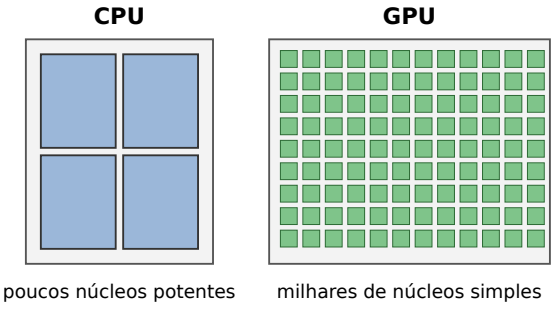
> **Ideia central**
> Todo o trabalho de computação gráfica é responder, para cada pixel: *que objeto aparece aqui, e de que cor?* O resto deste guia é como responder essas duas perguntas.

1.2 CPU e GPU

A **CPU** tem poucos núcleos, muito flexíveis. A **GPU** (placa de vídeo) tem milhares de núcleos simples (uma GTX 1080 tem 2560) que fazem **a mesma conta ao mesmo tempo** em muitos dados.

Isso combina com gráficos: uma cena tem milhares de vértices e milhões de pixels, e cada um recebe o mesmo cálculo (multiplicar por uma matriz, calcular a luz). A GPU faz isso em paralelo.

Os programas que rodam na GPU se chamam **shaders**. São pequenos e escritos em GLSL (no WebGL). Os dois que importam aqui são o **vertex shader** (roda uma vez por vértice) e o **fragment shader** (roda uma vez por pixel coberto).



1.3 O que o programa manda para a GPU

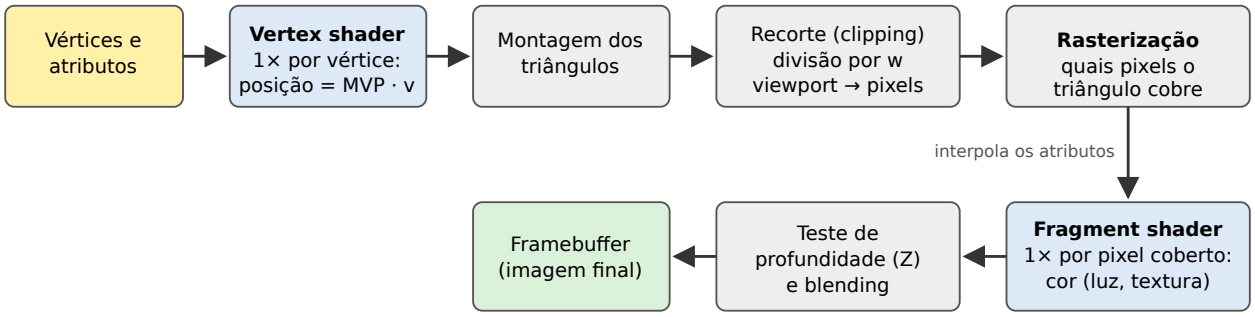
- **Vértices:** os pontos dos objetos. Cada vértice tem **atributos**: posição (x, y, z), cor, normal (para a luz), coordenada de textura (s, t).
- **Primitivas:** como ligar os vértices. Quase sempre **triângulos**: três pontos sempre formam um plano, o triângulo é sempre convexo e continua sendo triângulo depois de qualquer transformação. Um quadrado são 2 triângulos (6 vértices); um cubo, 12 triângulos (36 vértices).
- **Buffers:** listas com os atributos. Um buffer de posições, um de cores, um de normais... O vértice número *k* usa a entrada *k* de cada buffer. Todos precisam ter o mesmo número de entradas.
- **Uniforms:** valores iguais para todos os vértices de um desenho: as matrizes (M, V, P), a posição da luz, a posição da câmera.
- **Texturas:** imagens que ficam na memória da placa e são "coladas" nas superfícies.

Um vértice, muitos atributos

Se o mesmo ponto do espaço precisa de duas cores, duas normais ou duas coordenadas de textura diferentes (por exemplo, o canto de um cubo, que pertence a 3 faces), ele tem de ser **repetido** na lista de vértices. Por isso um cubo texturizado tem 24 vértices (4 por face) e não 8.

1.4 O pipeline gráfico, passo a passo

O **pipeline** é a linha de montagem que transforma vértices em pixels. Cada etapa recebe o resultado da anterior.



As caixas azuis são programáveis (shaders). As cinzas são fixas, feitas pelo hardware.

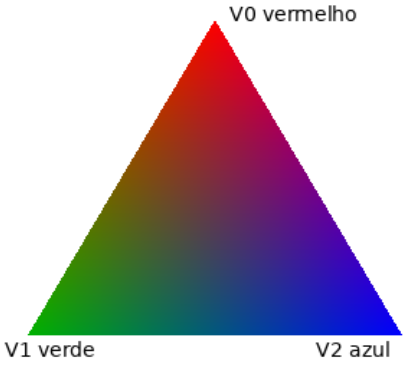
1. **Vertex shader** (uma vez por vértice): calcula onde o vértice vai parar. É aqui que entra a matriz **MVP** (Parte 4). Também pode calcular luz por vértice (Gouraud) e repassar atributos (cor, normal, coordenada de textura).
2. **Montagem:** junta os vértices de 3 em 3 em triângulos.
3. **Recorte, divisão por w e viewport:** joga fora o que está fora do campo de visão, corta triângulos que estão parcialmente fora e converte as coordenadas para pixels da tela (Parte 4.4 e 4.5).
4. **Rasterização:** descobre quais pixels cada triângulo cobre. Cada pixel coberto vira um **fragmento**. Aqui a GPU **interpola** os atributos dos 3 vértices para cada fragmento (seção 1.5).
5. **Fragment shader** (uma vez por fragmento): decide a cor. Pode ler textura e calcular luz por pixel (Phong).
6. **Teste de profundidade e blending:** o Z-buffer compara a profundidade do fragmento com a que já está no pixel. Se ele está atrás, é descartado; se está na frente, a cor é escrita. Blending mistura cores de objetos transparentes.

Na prova

Muitas questões perguntam "onde" algo acontece. Calcular a luz no **vertex shader** é sombreamento de **Gouraud**; no **fragment shader** é sombreamento de **Phong**. A interpolação de cores e coordenadas de textura acontece na **rasterização**, entre os dois shaders.

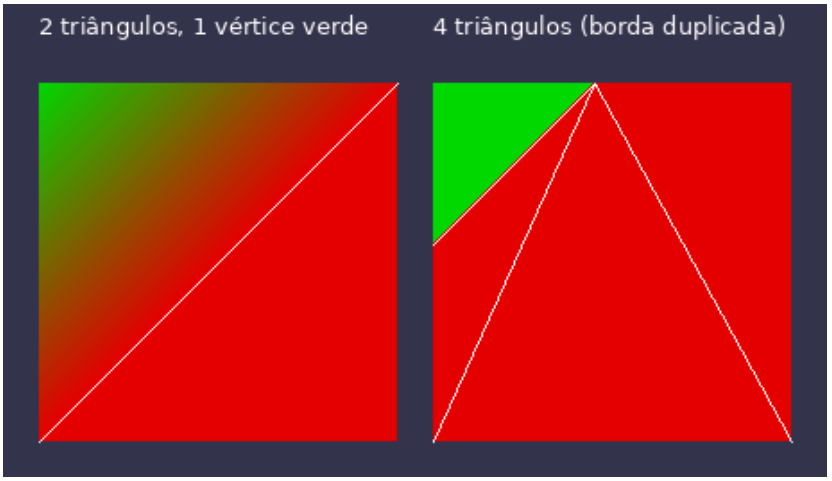
1.5 Interpolação de atributos: por que aparece degradê

O fragment shader recebe, para cada pixel, uma **média ponderada** dos valores dos 3 vértices do triângulo. O peso de cada vértice é maior quanto mais perto o pixel está dele (coordenadas **baricêntricas**). Se os vértices têm cores diferentes, o interior vira um degradê:



Um triângulo com um vértice vermelho, um verde e um azul. Nenhum pixel do interior tem cor "pura".

Consequência importante: **dentro de um triângulo não existe borda nítida de cor**. Para ter uma região com cor sólida e borda reta, a borda precisa ser uma **aresta** da malha, e os triângulos de cada lado precisam ter todos os vértices com a mesma cor.



Esquerda: 6 cores enviadas, 1 verde e 5 vermelhas → degradê. Direita: o quadrado foi subdividido; os pontos da fronteira entram duas vezes (uma cópia verde, uma vermelha).

> **Na prova (Prova 1, questão 5)**
> "Explique o resultado e o que fazer para conseguir o desejado." **Resposta:** a cor é atributo de vértice e é interpolada na rasterização; com 1 vértice verde e os outros vermelhos sai degradê. Solução: criar vértices sobre a fronteira desejada e subdividir o quadrado (1 triângulo todo verde + 3 todo vermelhos = 4 triângulos, 12 vértices), duplicando os vértices da fronteira com cores diferentes.

A mesma interpolação vale para **normais** (é o que faz o sombreamento de Phong) e para **coordenadas de textura** (é o que permite "colar" a imagem).

1.6 Lendo trechos de código que aparecem na prova

As provas mostram trechos de código só para dar contexto. O que cada termo significa:

Termo	Significado
`gl_Position = mvpMatrix * position`	Posição final do vértice (espaço de recorte). É a saída obrigatória do vertex shader.
`in` / `out`	Valores que entram e saem de um shader. O `out` do vertex shader chega **interpolado** como `in` no fragment shader.
`uniform`	Valor igual para todos: matrizes, posição da luz, câmera.
`texture(textureMap, coord)`	Lê a cor da textura na coordenada (s, t).
`normalize(v)`, `dot(a,b)`, `cross(a,b)`	Deixar com comprimento 1, produto escalar, produto vetorial (Parte 2).
`reflect(L, N)`, `max(x, 0.0)`, `pow(x, n)`	Reflete o vetor L na superfície de normal N; corta valores negativos; potência (brilho especular).
`bufferData`, `vertexAttribPointer`	Envia uma lista de atributos e diz como ler (quantos números por vértice).
`drawArrays(gl.TRIANGLES, 0, n)`	Desenha n vértices, de 3 em 3, como triângulos.
`texParameteri(..., MIN_FILTER / MAG_FILTER / WRAP_S ...)`	Filtro usado ao diminuir/aumentar a textura e o que fazer fora de [0,1] (Parte 6).
`translate`, `rotateX/Y/Z`, `scale`, `mult`	Funções do MV.js que criam matrizes e multiplicam (Parte 3).
`lookAt(eye, at, up)`, `perspective(fovy, aspect, near, far)`	Criam a matriz da câmera (V) e a de projeção (P) (Parte 4).

Parte 2 — A matemática necessária

Só quatro ferramentas aparecem nas provas: vetores, produto escalar, produto vetorial e multiplicação de matrizes 4×4.

2.1 Pontos e vetores

- Um **ponto** é uma posição: P = (x, y, z).
- Um **vetor** é uma direção com tamanho: v = (vx, vy, vz). A diferença entre dois pontos é um vetor: `B − A = (Bx−Ax, By−Ay, Bz−Az)`, que vai de A até B.
- **Comprimento:** `|v| = √(vx² + vy² + vz²)`
- **Normalizar** é dividir pelo comprimento, para ficar com tamanho 1 (vetor unitário ou versor): `v̂ = v / |v|`. Direções de luz, câmera e normais são sempre normalizadas antes de usar.

> **Exemplo**
> v = (3, 0, 4) → |v| = √(9 + 0 + 16) = 5 → v̂ = (0,6 ; 0 ; 0,8).

2.2 Produto escalar (dot)

```
a · b = ax·bx + ay·by + az·bz = |a|·|b|·cos θ
```

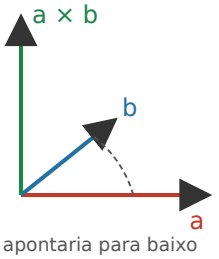
Se os dois vetores são unitários, o resultado é o **cosseno do ângulo** entre eles:

- 1 → mesma direção; 0 → perpendiculares; −1 → opostos; negativo → mais de 90°.
- É o coração da iluminação: a luz difusa é proporcional a N · L (Parte 5).
- O sinal diz "de que lado" algo está. Se N · L < 0, a luz está atrás da superfície.

2.3 Produto vetorial (cross)

```
a × b = (ay·bz − az·by , az·bx − ax·bz ,
ax·by − ay·bx)
```

- O resultado é **perpendicular** a a e a b. Por isso é usado para calcular a **normal** de uma face: o produto de duas arestas.
- A direção segue a **regra da mão direita**: dedos de a para b, o polegar aponta para a × b.
- **A ordem importa:** b × a = −(a × b). Trocar a ordem **inverte a normal**.
- Na câmera, o produto vetorial é usado para montar os eixos (Parte 4.2).



b × a apontaria para baixo

> **Exemplo (face da frente de um cubo)**
> A = (−1,−1,1), B = (1,−1,1), C = (1,1,1).
> (B − A) × (C − B) = (2,0,0) × (0,2,0) = (0·0 − 0·2 ; 0·0 − 2·0 ; 2·2 − 0·0) = (0, 0, 4) → normalizado (0, 0, 1): aponta **para fora** do cubo. Certo.
> Na ordem trocada, (C − B) × (B − A) = (0, 0, −4): aponta **para dentro**. Com essa normal, N · L fica negativo e a face sai preta.

2.4 Matrizes e multiplicação

Matriz × vetor: cada componente do resultado é o produto escalar de uma **linha** da matriz com o vetor.

```
[ a b c d ]   [x]   [ a·x + b·y + c·z + d·w ]
[ e f g h ] · [y] = [ e·x + f·y + g·z + h·w ]
[ i j k l ]   [z]   [ i·x + j·y + k·z + l·w ]
[ m n o p ]   [w]   [ m·x + n·y + o·z + p·w ]
```

Matriz × matriz: o elemento da linha i, coluna j do resultado é **linha i da primeira · coluna j da segunda**. No LibreOffice ou Excel, a função é `MMULT` (Parte 8).

- **Não é comutativa:** A·B ≠ B·A em geral. A ordem das transformações muda o resultado.
- **É associativa:** (A·B)·C = A·(B·C). Por isso dá para juntar várias transformações em uma única matriz e aplicar essa matriz a todos os vértices (é muito mais barato).
- **Identidade I:** 1 na diagonal e 0 no resto; I·v = v.
- **Inversa M^{-1}:** desfaz M (M^{-1}·M = I). Para inverter um produto, inverta cada uma e troque a ordem: $(A \cdot B)^{-1} = B^{-1} \cdot A^{-1}$.

2.5 Coordenadas homogêneas: por que 4×4?

Rotação e escala são multiplicações (x' = a·x + b·y + …), mas a translação é uma **soma** (x' = x + dx). Uma matriz 3×3 não consegue somar uma constante. O truque é acrescentar uma 4ª coordenada, **w**:

- **Ponto:** (x, y, z, **1**). A última coluna da matriz (dx, dy, dz) é somada, porque é multiplicada por w = 1.
- **Vetor/direção** (normal, direção da luz): (x, y, z, **0**). A translação é ignorada, o que é certo: mover um objeto não muda para onde suas faces apontam.

```
T(dx,dy,dz) · (x, y, z, 1) = (x+dx, y+dy, z+dz, 1)
[1 0 0 dx]
[0 1 0 dy]
[0 0 1 dz]
[0 0 0 1 ]
```

> **Cuidado com a normal**
>
> Uma normal guardada como vec4 precisa ter **w = 0**. No código da aula isso é feito com `normal[3] = 0`. Com w = 1, a translação da matriz de modelo "entra" na normal e a iluminação fica errada.

Depois da projeção (Parte 4), w deixa de ser 1. Para voltar a um ponto normal, divide-se tudo por w: (x/w, y/w, z/w). Isso se chama **divisão perspectiva** ou homogeneização.

Parte 3 — Transformações geométricas

Toda transformação é aplicada **a cada vértice** do objeto: o objeto inteiro se move porque todos os seus pontos se movem. Na convenção da disciplina (MV.js, OpenGL), o ponto é uma coluna e a matriz fica à esquerda: `P' = M · P`.

3.1 Translação, escala e rotação

Translação T(dx, dy, dz)	Escala S(sx, sy, sz)

`[1 0 0 dx]`
`[0 1 0 dy]`
`[0 0 1 dz]`
`[0 0 0 1 ]`
Desloca o objeto.
	`[sx 0 0 0]`
	`[0 sy 0 0]`
	`[0 0 sz 0]`
	`[0 0 0 1]`
	Aumenta/diminui **em relação à origem**. Escala uniforme: sx = sy = sz. Fator negativo espelha.

Rotação em X: Rx(θ)	Rotação em Y: Ry(θ)	Rotação em Z: Rz(θ)

`[1 0 0 0]`
`[0 cos -sen 0]`
`[0 sen cos 0]`
`[0 0 0 1]`
	`[ cos 0 sen 0]`
	`[ 0 1 0 0]`
	`[-sen 0 cos 0]`
	`[ 0 0 0 1]`
		`[cos -sen 0 0]`
		`[sen cos 0 0]`
		`[ 0 0 1 0]`
		`[ 0 0 0 1]`

- A coordenada do eixo de rotação não muda (em Rz, o z fica igual).
- Ângulo positivo = sentido **anti-horário**, olhando do lado positivo do eixo para a origem. Ângulo negativo gira no sentido horário.
- Toda rotação é **em torno da origem**. Para girar em torno de outro ponto, veja 3.4.
- Valores úteis: cos 30° = 0,8660, sen 30° = 0,5; cos 45° = sen 45° = 0,7071; cos 60° = 0,5, sen 60° = 0,8660; cos 90° = 0, sen 90° = 1.

> **Graus e radianos**
> Funções de planilha (COS, SEN) e de programação usam **radianos**. Use `COS(RADIANS(60))`, não `COS(60)`. Erro clássico.

3.2 Inversas

- $T(d)^{-1} = T(-d)$: desloca de volta.
- $S(s)^{-1} = S(1/s)$.
- $R(\theta)^{-1} = R(-\theta) = R^{T}$ (a transposta; vale para rotações).

3.3 Composição: a ordem importa

Para aplicar primeiro T_1, depois T_2, depois T_3, a matriz composta é $M = T_3 \cdot T_2 \cdot T_1$, multiplicada **na ordem inversa**. A transformação mais à direita é a primeira a agir sobre o ponto.

> **Exemplo: girar 90° em Z e transladar 2 em X, ponto P = (1, 0, 0)**
> **Primeiro gira, depois translada**, M = T · R: R·P = (0, 1, 0) → T → **(2, 1, 0)**.
> **Primeiro translada, depois gira**, M = R · T: T·P = (3, 0, 0) → R → **(0, 3, 0)**.
> Mesmas matrizes, resultados diferentes.

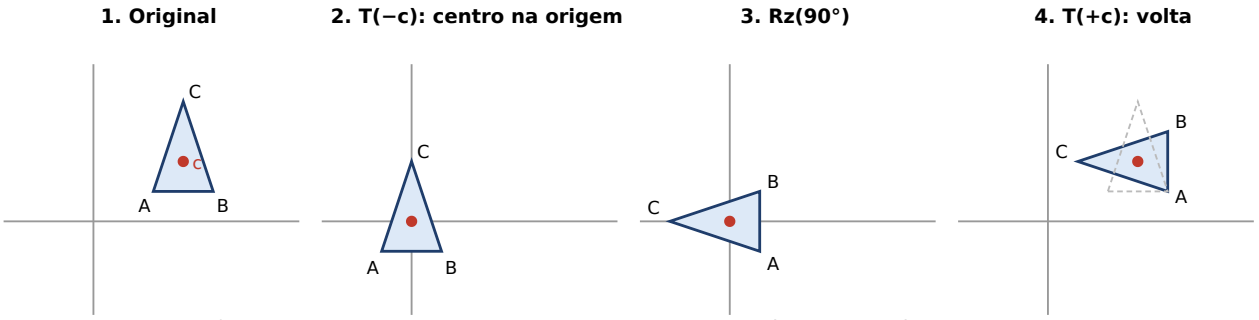
3.4 Girar ou escalar em torno de um ponto (o centro do objeto)

Rotação e escala sempre agem em torno da origem. Se o objeto está longe da origem, ele "orbita" em vez de girar no lugar. A solução é uma composição de três passos:

1. **T(−c):** leva o centro c para a origem.
2. **R** (ou S): gira (ou escala), agora em torno da origem.
3. **T(+c):** devolve o centro para onde estava.

`M = T(c) · R · T(−c)` . O centro de um triângulo é o **centróide**, a média dos vértices:

`c = ((Ax+Bx+Cx)/3 , (Ay+By+Cy)/3 , (Az+Bz+Cz)/3)` .



Triângulo A(2,1), B(4,1), C(3,4), centro c = (3,2). O tracejado no último quadro é a posição original.

> **Exemplo numérico (2D, girando 90°)**
> c = (3, 2). A − c = (−1, −1) → girado 90° (x, y) → (−y, x) = (1, −1) → + c = **A' = (4, 1)**. Da mesma forma, B' = (4, 3) e C' = (1, 2).
> Se girasse **em torno da origem** (sem T(±c)): A' = (−1, 2). O triângulo iria parar em outro lugar.

A escala em torno do centro é igual: `M = T(c) · S · T(−c)` . Um bom teste: depois de T(c)·R·T(−c), o centróide continua no mesmo lugar.

3.5 Traduzindo o enunciado em matrizes

O enunciado diz…	Matriz
"Rotação em torno do eixo Z por 45°, considerando a origem"	Rz(45°)
"Rotação em torno do eixo Z central", "em relação ao ponto central do triângulo"	T(c) · Rz(θ) · T(−c)
"Rotação em X por −60°, considerando o próprio eixo"	T(c) · Rx(−60°) · T(−c): eixo paralelo a X passando pelo centro
"Aumento do objeto em dobro", "aumento da escala em 2"	S(2). O critério usado nas resoluções é em relação à origem (eixo de coordenadas). Se o professor quiser no centro: T(c)·S(2)·T(−c)
"Mudança da escala em 0,5"	S(0,5)
"Deslocamento no eixo X em 2"	T(2, 0, 0)

Montando M: liste as transformações na ordem do enunciado (T_1, T_2, T_3…) e multiplique de trás para frente: `M = T₃ · T₂ · T₁` . Exemplo da Prova 1 (triângulo U): rotação central em X, depois escala 2, depois deslocamento em X:

`M = T(2,0,0) · S(2) · [T(c) · Rx(−60°) · T(−c)]`

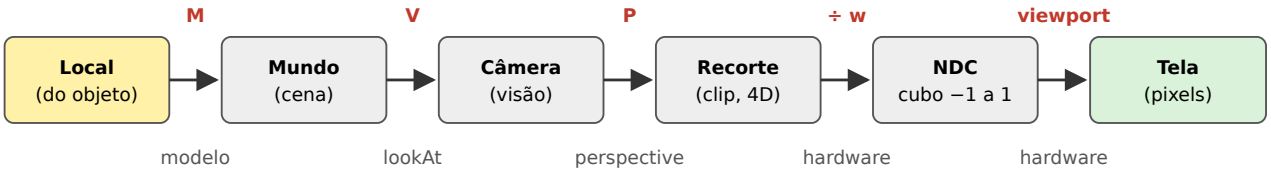
Na prova

A questão pede "todas as matrizes intermediárias, com nomes". Escreva cada uma separada e com nome (T1 = translação para a origem, R = rotação, T2 = translação de volta, S = escala, M = modelo, V = visualização, P = projeção, MVP = final). Depois mostre os produtos. A planilha (Parte 8) faz exatamente isso.

Parte 4 — Câmera, projeção e a matriz MVP

4.1 A viagem de um vértice

Cada vértice passa por uma sequência de **espaços de coordenadas**. Cada passagem é uma matriz 4×4 (menos a divisão por w e o viewport, que são feitos pelo hardware).



Clip = **P · V · M** · vértice → MVP = P · V · M (a ordem é da direita para a esquerda)

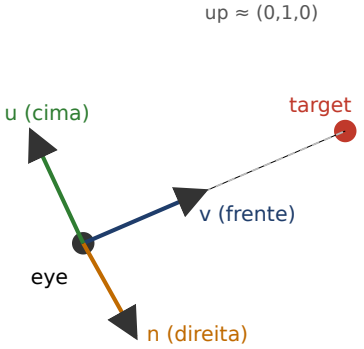
- **M (modelo):** coloca o objeto na cena (Parte 3).
- **V (visualização):** reescreve tudo do ponto de vista da câmera.
- **P (projeção):** aplica a perspectiva e define o volume visível.
- **Divisão por w:** termina a perspectiva; o que é visível fica dentro do cubo de −1 a 1 (NDC).
- **Viewport:** converte de −1..1 para pixels da janela.

4.2 A matriz de visualização (câmera, lookAt)

A câmera é definida por três dados: a posição **eye**, o ponto para onde ela olha (**target** ou **at**) e o vetor **up** (qual direção é "para cima", normalmente (0, 1, 0)).

Passo 1: os eixos da câmera

1. `v = normalize(target − eye)` : direção para onde a câmera olha ("frente").
2. `n = normalize(v × up)` : a "direita" da câmera.
3. `u = n × v` : o "cima" verdadeiro da câmera (já unitário).



Passo 2: a matriz V (as linhas são os eixos; a última coluna leva o eye para a origem)

```
    [ nx   ny   nz   −(n·eye) ]
V = [ ux   uy   uz   −(u·eye) ]
    [−vx  −vy  −vz    (v·eye) ]
    [ 0    0    0       1     ]
```

- A matriz V "move o mundo ao contrário": em vez de mover a câmera, ela desloca e gira a cena para que a câmera fique na origem, olhando para **−z**. Pontos na frente da câmera ficam com **z negativo**.
- É uma **mudança de coordenadas** (slides de Transformações): as linhas da matriz são os eixos da câmera escritos em coordenadas do mundo.
- O up não pode ser paralelo a v (a câmera olhando reto para cima ou para baixo): o produto vetorial daria zero.

Os nomes dos eixos mudam de material para material

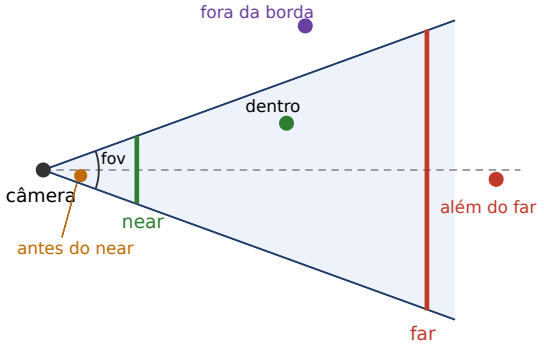
Fonte	Frente	Direita	Cima
Este guia, as planilhas e o MV.js	v	n	u
Planilha do professor (ExercícioViewProjection)	w	u	v
Slides (DirectX)	w	u	v
Livro	n	u	v

O cálculo é sempre o mesmo: frente = alvo − olho normalizado; direita = frente × up; cima = direita × frente.

4.3 A matriz de projeção perspectiva

A câmera enxerga um tronco de pirâmide chamado **frustum**, definido por quatro números:

- **fovy:** abertura vertical (campo de visão), em graus.
- **aspect:** largura/altura da janela (800×600 → 1,333).
- **near:** distância do plano mais próximo que aparece.
- **far:** distância do plano mais distante que aparece. Tudo além some.



Com `f = 1 / tan(fovy/2)` e d = far − near:

$$P = \begin{bmatrix} f/aspect & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & -(far+near)/d & -2\cdot far\cdot near/d \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

O que cada linha faz:

- Linhas 1 e 2: escalam x e y conforme o campo de visão. Quanto menor o fov, maior o f e mais "zoom".
- **Linha 4: w_clip = −z_câmera**, ou seja, a **distância** do ponto à frente da câmera. É por isso que, depois da divisão por w, objetos mais longe ficam menores. É a perspectiva (por semelhança de triângulos: y' = y·d/z).
- Linha 3: guarda a profundidade. Depois da divisão, o plano near vira z = −1 e o far vira z = +1. A profundidade não pode ser descartada: o Z-buffer precisa dela.

FOV horizontal (Prova 3)

Se o enunciado der o FOV **horizontal**, converta para o vertical:

`tan(fovx/2) = aspect · tan(fovy/2)` .

fovx = 120°, janela 2048×1024 (aspect 2): tan(fovy/2) = tan 60° / 2 = 1,732/2 = 0,866 → fovy/2 = 40,89° → **fovy = 81,79°** e f = 1/0,866 = 1,1547.

Duas convenções (não se assuste com o livro e os slides)

	Código da aula, MV.js, planilhas (OpenGL)	Slides de Projeção e livro (DirectX)
Câmera olha para	$-z$	$+z$
z depois da divisão	-1 (near) a $+1$ (far)	0 (near) a 1 (far)
Linha/coluna da perspectiva	-1 na última linha	1 na última coluna; termos $f/(f-n)$ e $-nf/(f-n)$
Multiplicação	$P \cdot V \cdot M \cdot$ ponto (ponto em coluna)	ponto $\cdot M \cdot V \cdot P$ (ponto em linha, nos slides)

O desenho final é o mesmo. Nas provas use a do código da aula e **escreva qual convenção está usando**.

Projeção ortogonal (livro 5.5.2): os raios são paralelos, o volume visível é uma caixa e não há encolhimento com a distância. A matriz só translada e escala a caixa para o cubo canônico, e w continua 1. Serve para desenho técnico; nos jogos e nas provas, a perspectiva é a que importa.

4.4 NDC e viewport: chegando aos pixels

Depois de multiplicar pela MVP, o ponto está no **espaço de recorte** (x, y, z, w). Dividindo por w obtém-se o **NDC** (coordenadas normalizadas do dispositivo): tudo que é visível tem x, y e z entre -1 e 1. A transformação de viewport leva isso para pixels, numa janela de largura W e altura H:

```
x_pixel = (x_ndc + 1) · W/2 y_pixel = (y_ndc + 1) · H/2
```

Isso com a origem no **canto inferior esquerdo**, como no OpenGL e no livro. Se a origem for o canto superior esquerdo, use $y_{pixel} = (1 - y_{ndc}) \cdot H/2$. O z vai para o Z-buffer.

Por que a proporção da janela importa

Se o aspect da projeção não for igual a largura/altura da janela, os objetos saem esticados (um círculo vira elipse). Por isso o aspect entra na matriz P.

4.5 O triângulo aparece? Recorte e oclusão

Um vértice está **dentro do volume visível** quando, no espaço de recorte:

```
w > 0 e −w ≤ x ≤ w, −w ≤ y ≤ w, −w ≤ z ≤ w
```

Isso equivale a todas as coordenadas NDC entre -1 e 1, com w positivo. Para cada vértice que falha, diga o porquê:

O que aparece no cálculo	Significado
$w \le 0$	O ponto está **atrás da câmera**. O NDC fica sem sentido (o sinal inverte).
$z_{ndc} > 1$	**Além do far**: longe demais.
$z_{ndc} < -1$	**Antes do near**: perto demais.
$x_{ndc} < -1$ ou > 1	Fora pela esquerda ou pela direita.
$y_{ndc} < -1$ ou > 1	Fora por baixo ou por cima.

Classificando o triângulo:

- **3 vértices dentro:** totalmente dentro do volume. Aparece inteiro, se nada estiver na frente.

- **1 ou 2 dentro: parcialmente visível**. O recorte (clipping) corta o triângulo na borda do volume e só a parte de dentro é desenhada.
- **Nenhum dentro:**
  - Se os 3 estão fora pelo **mesmo lado** (os 3 além do far, ou os 3 com x_ndc > 1…), o triângulo inteiro está fora, porque ele é convexo. **Invisível.**
  - Se estão fora por **lados diferentes** (por exemplo, 2 além do far e 1 abaixo da janela), o meio do triângulo ainda pode cruzar o volume, e um pedaço aparece (aconteceu na Prova 3).

> **"Fora do volume" não é "ocluso"**
> **Ocluso** quer dizer que está dentro do volume mas **atrás de outro objeto**: o Z-buffer descarta seus fragmentos porque já existe algo mais perto no mesmo pixel. Um triângulo além do far ou atrás da câmera está **fora do volume**, não ocluso.

Por que o recorte é feito antes da divisão por w (livro 5.5.3.4): um ponto atrás da câmera tem w negativo; ao dividir, os sinais se invertem e ele "reapareceria" na frente. Recortar antes evita isso.

Culling (slides): o hardware também pode descartar faces viradas para trás (back-face culling) e objetos totalmente fora do campo de visão, só para economizar trabalho.

4.6 Mudando a câmera de propósito

Objetivo	O que mexer
Objeto além do far	Aumentar o far ou aproximar a câmera.
Objeto saindo pelas bordas	Afastar a câmera (na mesma direção de visão), aumentar o fov, ou mirar o target no centro do objeto.
Objeto cortado pelo near	Afastar a câmera ou diminuir o near (que precisa continuar > 0).
Dois objetos dentro do volume, mas **só um visível**	Colocar a câmera sobre a reta que liga os centros dos dois, do lado do objeto **maior**. O maior fica na frente e cobre o menor (oclusão pelo Z-buffer). Confira que os dois cabem no frustum (aumente o far se precisar).

4.7 Exemplo completo, feito à mão

> **Enunciado**
> Ponto P = (1, 1, 0). Transformações: rotação de 90° em Z (em torno da origem), depois escala 2. Câmera em eye = (0, 0, 5), olhando para a origem, up = (0, 1, 0). Janela 800×600, fovy = 90°, near = 1, far = 10.

1. Matriz de modelo. M = S(2) · Rz(90°). Com cos 90° = 0 e sen 90° = 1:

```
Rz(90°)          S(2)           M = S·Rz
[0 -1 0 0]      [2 0 0 0]       [0 -2 0 0]
[1  0 0 0]      [0 2 0 0]       [2  0 0 0]
[0  0 1 0]      [0 0 2 0]       [0  0 2 0]
[0  0 0 1]      [0 0 0 1]       [0  0 0 1]
```

Mundo: M · (1, 1, 0, 1) = (0·1 − 2·1 ; 2·1 + 0 ; 0 ; 1) = **(−2, 2, 0)**.

2. Câmera. v = normalize((0,0,0) − (0,0,5)) = (0, 0, −1). n = v × up = (0,0,−1) × (0,1,0) = (0·0 − (−1)·1 ; (−1)·0 − 0·0 ; 0·1 − 0·0) = (1, 0, 0). u = n × v = (0, 1, 0).
Translação: −(n·eye) = 0; −(u·eye) = 0; v·eye = −5.

Câmera: V · (−2, 2, 0, 1) = **(−2, 2, −5)**. O ponto está 5 unidades à frente da câmera (z negativo = na frente).

```
V
[1 0 0  0]
[0 1 0  0]
[0 0 1 -5]
[0 0 0  1]
```

3. Projeção. f = 1/tan 45° = 1; aspect = 800/600 = 1,3333 → f/aspect = 0,75; d = 9; −(10+1)/9 = −1,2222; −2·10·1/9 = −2,2222.

```
P
[0.75 0  0       0     ]
[0    1  0       0     ]
[0    0 -1.2222 -2.2222]
[0    0 -1       0     ]
```

```
MVP = P · V · M
[0  -1.5  0       0     ]
[2   0    0       0     ]
[0   0   -2.4444  3.8889]
[0   0   -2       5     ]
```

4. Recorte. MVP · (1, 1, 0, 1) = (−1,5 ; 2 ; 3,8889 ; 5). Como w = 5 > 0 e |x|, |y|, |z| ≤ 5, o ponto está **dentro**.

5. NDC. Dividindo por w = 5: (−0,3 ; 0,4 ; 0,7778).

6. Pixel. x = (−0,3 + 1)·400 = **280**; y = (0,4 + 1)·300 = **420**.

> **Dois contraexemplos com a mesma câmera**
> Ponto do mundo (0, 0, −6): na câmera fica em z = −11, a 11 unidades (far = 10). Clip = (0 ; 0 ; 11,22 ; 11) → z_ndc = 1,02 > 1 → **além do far**.
> Ponto do mundo (0, 0, 6): na câmera fica em z = +1, atrás dela. Clip w = −1 → **atrás da câmera**.

4.8 Receita da questão de MVP

1. Leia os dígitos da matrícula e substitua X e Y nos vértices. Na Prova 1 há uma regra: Y = (dígito mod 5) + 2, X = 8 − Y.
2. Calcule o centróide c, se houver rotação ou escala "central".
3. Escreva cada matriz intermediária com nome: T1 = T(−c), R, T2 = T(c), S, T(desloc.).
4. Monte M multiplicando na ordem inversa do enunciado (3.5).
5. Calcule a posição de cada vértice no mundo: M · vértice. Confira: numa rotação central, o centróide não muda.
6. Defina o target. Se for "o centro do triângulo", use o centro já transformado (e diga isso).
7. Calcule v, n, u e monte V.
8. Calcule f (converta o FOV, se ele for horizontal), aspect e d, e monte P.
9. MVP = P · V · M. Essa é a "matriz final".
10. Para cada vértice: clip = MVP · (x, y, z, 1).
11. NDC = clip / w. Pixel pela fórmula de viewport.
12. Classifique cada vértice (tabela da 4.5) e conclua: visível, parcialmente visível (diga qual plano corta) ou invisível (diga por quê).
13. Se pedirem para mudar a câmera, use a tabela da 4.6 e **confira de novo** com as novas matrizes.

Parte 5 — Iluminação e sombreamento

Dois conceitos que a prova separa com cuidado:

- **Modelo de iluminação (ou de reflexão):** a *fórmula* que dá a cor de **um ponto** a partir da luz, da normal e da câmera. Aqui é o modelo de **Phong** (ou Blinn-Phong).
- **Sombreamento (shading):** *em quais pontos* a fórmula é calculada e como o resto é preenchido: uma vez por polígono (constante), nos vértices (Gouraud) ou em cada pixel (Phong).

> **Na prova (Prova 2)**
> "Em cada plano foi aplicado o **mesmo modelo de iluminação**. Explique os resultados diferentes." A diferença está no **sombreamento**, não no modelo. Veja a 5.4.

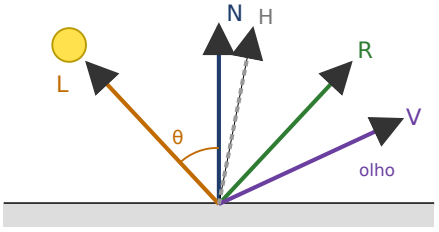
5.1 Fontes de luz

- **Iluminação local:** só conta a luz que vem direto das fontes (é o que a GPU faz em tempo real). **Global:** conta também a luz refletida por outros objetos (ray tracing; caro).
- **Luz ambiente:** sem posição nem direção, ilumina tudo igual. É uma aproximação grosseira da luz que vem rebatida do ambiente. Sem ela, o que não recebe luz direta fica totalmente preto.
- **Direcional:** raios paralelos, como o sol. Só tem direção.
- **Pontual:** tem posição e emite para todos os lados, como uma lâmpada. Pode enfraquecer com a distância.
- **Spot:** como a pontual, mas só dentro de um cone, como uma lanterna.

5.2 O modelo de Phong

Num ponto da superfície, quatro vetores unitários:

- **N:** a normal (para onde a superfície "olha").
- **L:** do ponto **para a luz**.
- **V:** do ponto **para o olho** (câmera).
- **R:** o reflexo de L em torno de N: `R = 2(N·L)N − L`.



A cor é a soma de três parcelas:

```
I = Ia·Ka + Kd·I·max(N·L, 0) +
Ks·I·max(R·V, 0)ⁿ
```

Parcela	Fórmula	O que representa
Ambiente	Ia·Ka	Luz constante que vem de todo lado. Igual em todo o objeto.
Difusa (Lambert)	Kd·I·max(N·L, 0)	Superfície fosca, que espalha a luz igual para todos os lados. Mais forte quando a luz bate de frente (cos θ = 1) e zero quando bate de lado ou por trás. **Não depende de onde está o olho.**
Especular	Ks·I·max(R·V, 0)ⁿ	O brilho (reflexo da luz). Só aparece quando o olho está perto da direção de reflexão R. **Depende do olho.**

- Ka, Kd, Ks (entre 0 e 1) são propriedades do **material**: quanto ele reflete de cada tipo.
- **n (brilho, shininess):** quanto maior, menor e mais concentrado o reflexo (metal polido: 100 a 500); n pequeno dá um brilho largo e espalhado.
- O **max(..., 0)** evita luz negativa quando a luz está atrás da superfície.

- **Aproximação de Blinn:** em vez de R, usa-se `H = normalize(L + V)` (vetor do meio) e $(N \cdot H)^n$. É mais barata e dá resultado parecido (com outro expoente). Esse é o modelo **Blinn-Phong** do livro.
- Com cor, a fórmula é aplicada separada para R, G e B. Com várias luzes, as contribuições se somam. A intensidade pode ser dividida por uma função da distância (atenuação), mas a ambiente não atenua.

Exemplo numérico

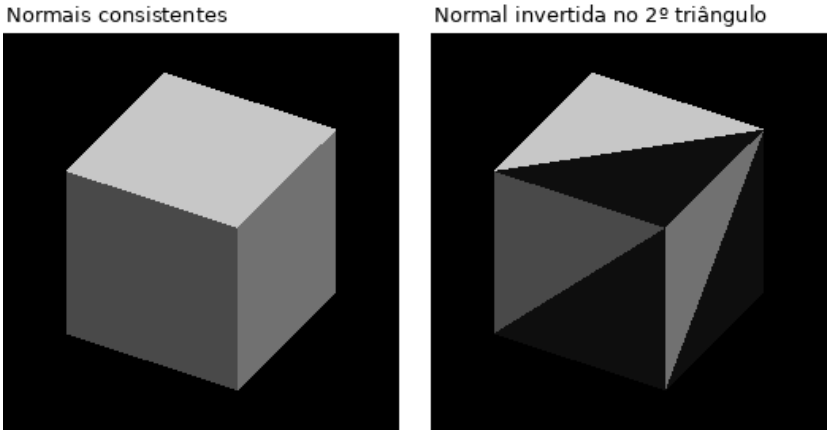
Ponto na origem, N = (0, 1, 0). Luz em (0, 2, 2) → L = (0, 2, 2)/2,83 = (0 ; 0,707 ; 0,707). Ia = 0,2, Ka = 1, I = 1, Kd = 0,8, Ks = 0,5, n = 10.
Ambiente = 0,2. Difusa: N·L = 0,707 → 0,8 · 0,707 = 0,566.
R = 2(0,707)(0,1,0) − (0 ; 0,707 ; 0,707) = (0 ; 0,707 ; −0,707).
Olho em (0, 2, −2): V = (0 ; 0,707 ; −0,707) = R → R·V = 1 → especular = 0,5. É o brilho máximo.
Olho em (0, 2, 0): V = (0, 1, 0) → R·V = 0,707 → $0{,}707^{10}$ = 0,031 → especular = 0,016. Quase nada: basta sair um pouco da direção R e o brilho some.

O sentido de L

Na teoria, L vai **do ponto para a luz**. Se o código calcular `posição − posiçãoDaLuz` (da luz para o ponto) e usar no termo difuso, o sinal de N·L inverte: as faces viradas para a luz ficam escuras e as de trás claras. Isso afeta sempre a **face inteira**, porque N é constante na face. Nunca divide uma face ao meio.

5.3 Normais

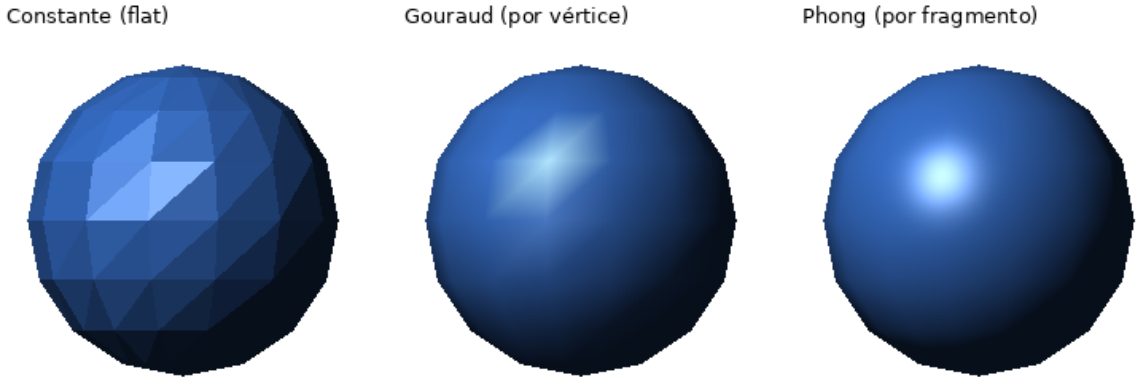
- **Normal de face:** produto vetorial de duas arestas do polígono (2.3). Os vértices de cada face precisam estar numa **ordem consistente** (anti-horária vista de fora), para que todas as normais apontem para fora.
- **Normal de vértice:** média das normais das faces que compartilham o vértice, normalizada: n = normalize($n_1 + n_2 + n_3 + n_4$). Isso dá aparência lisa (esfera). Para um cubo, que tem quinas, usa-se a normal da face em cada um dos 4 vértices da face, e o vértice é repetido (1.3).
- A normal é transformada pela matriz de modelo como **vetor** (w = 0), porque só deve girar. Com escala não uniforme, o certo é usar a inversa transposta (no MV.js, `normalMatrix`).
- Depois de interpolada, a normal perde o comprimento 1 e precisa ser **normalizada de novo** no fragment shader.



Se os dois triângulos de uma face recebem normais opostas (ordem dos vértices trocada no produto vetorial), um deles fica com N·L < 0 e sai preto: a face aparece cortada na diagonal (Prova 1, questão 3).

Na prova (Prova 1, questão 3)
O "Gepeto" (ChatGPT) culpou o sinal do vetor de luz e as "normais por face". As duas explicações estão erradas. Sinal da luz invertido afeta **faces inteiras**; uma normal por face é **correta** para um cubo e dá cor constante, sem regiões triangulares. O corte diagonal mostra que os **dois triângulos da mesma face têm normais diferentes**: erro no cálculo da normal (ordem dos vértices no produto vetorial). Correção: uma normal por quad, com ordem consistente, repetida nos 6 vértices, e w = 0.

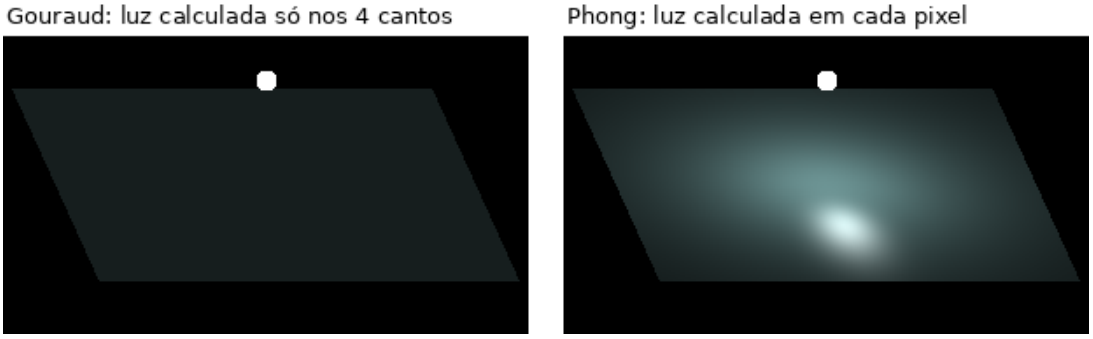
5.4 Sombreamento: constante, Gouraud e Phong



A mesma esfera de poucos polígonos, com o mesmo modelo de iluminação e três sombreamentos diferentes.

	Constante (flat)	Gouraud	Phong
Onde a luz é calculada	Uma vez por polígono	Nos vértices (vertex shader)	Em cada pixel (fragment shader)
O que é interpolado	Nada	A **cor**	A **normal** (e a posição)
Aparência	Facetada; bandas de Mach	Lisa; brilho deformado ou perdido	Lisa; brilho correto e redondo
Custo	Mínimo	Proporcional ao número de vértices	Proporcional ao número de pixels (mais caro)

- **Bandas de Mach:** o olho humano exagera a diferença de brilho na fronteira entre duas áreas de cor constante. O lado escuro parece mais escuro e o claro mais claro, o que realça as facetas do sombreamento constante.
- **Por que o Gouraud perde o brilho:** o reflexo especular é estreito. Se ele cai no meio de um polígono, longe dos vértices, nenhum vértice "vê" o brilho; a interpolação só mistura as cores dos vértices, que são apagadas. Se cai exatamente num vértice, o brilho se espalha pelos triângulos vizinhos e fica deformado.



Um plano grande (2 triângulos) com uma luz pontual em cima, como na Prova 2. Os 4 cantos estão longe da luz, então o Gouraud dá um plano quase uniforme e apagado. O Phong mostra a área clara sob a luz e o brilho especular.

> **Na prova (Prova 2): resposta modelo**
> Mesmo modelo (Phong: ambiente + difusa + especular), sombreamentos diferentes. Esquerda: **Gouraud**; a luz é calculada só nos 4 vértices, longe da fonte, e as cores são interpoladas, então o brilho do centro se perde. Direita: **Phong**; a normal é interpolada e a luz calculada em cada fragmento, então aparece a mancha clara sob a luz.
> **Gouraud:** vantagem, rápido (conta por vértice); desvantagem, perde ou deforma brilhos e depende de tesselação fina.
> **Phong:** vantagem, realista mesmo com polígonos grandes; desvantagem, mais caro (conta por pixel e normaliza a normal).

Limitações de todo sombreamento interpolado (livro 7.3.4):

- **Silhueta poligonal:** o contorno continua mostrando os polígonos.
- **Distorção de perspectiva:** a interpolação é feita na tela, onde o ponto do meio não corresponde ao meio real (o mesmo problema das texturas, 6.5).
- **Orientação do polígono:** o mesmo polígono girado pode ser sombreado diferente.
- **Vértices não compartilhados:** um vértice no meio da aresta de um polígono vizinho cria descontinuidades.
- **Normais de vértice incorretas:** uma média que "alisa" o que deveria ser quina.

Quase todas diminuem com polígonos menores (tesselação mais fina), ao custo de desempenho.

5.5 Sombras (visão geral)

- **Sombra dura:** o ponto vê ou não vê a luz. **Sombra suave:** a luz tem tamanho, então há a **umbra** (totalmente na sombra) e a **penumbra** (parcialmente).
- **Shadow volumes:** o volume de sombra de cada objeto é construído como geometria e contado no *stencil buffer*. Sombras exatas, mas lento.
- **Shadow mapping** (o mais usado em jogos), em duas passadas:
  1. Renderiza a cena **do ponto de vista da luz**, guardando só a profundidade (o mapa de sombra).
  2. Renderiza da câmera. Para cada ponto, compara a distância até a luz com a profundidade guardada; se for maior, há algo na frente e o ponto está na sombra.

  A luz direcional usa projeção ortogonal, a spot usa perspectiva e a pontual usa um cube map (6 vistas).
- **Problemas do shadow mapping:** serrilhado (o mapa tem resolução limitada) e "acne" de sombra (erros de precisão). Corrige-se com **depth bias**: pouco bias causa z-fighting; bias demais faz a luz vazar.
- **Melhorias: cascaded shadow maps** (vários mapas, mais resolução perto da câmera); **PCF** (compara com vários vizinhos e faz a média, para bordas suaves); **variance shadow maps** (sombras bem suaves, com risco de vazamento de luz).

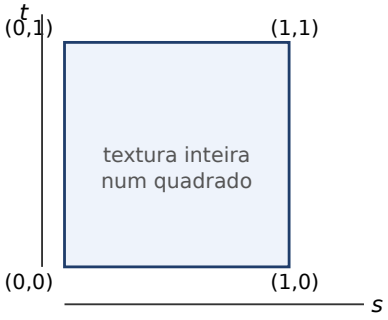
Parte 6 — Texturas

6.1 Conceitos

- **Textura** é uma imagem guardada na memória da placa e "colada" numa superfície. Cada pixel da textura se chama **texel**.
- A posição dentro da textura usa **coordenadas de textura** (s, t), também chamadas (u, v), que vão de **0 a 1** qualquer que seja a resolução da imagem. Assim a mesma coordenada funciona para uma textura de 64×64 ou 2048×2048.
- Nas provas, a **origem (0, 0) é o canto inferior esquerdo**: s cresce para a direita e t para cima. (Arquivos de imagem guardam a linha de cima primeiro; por isso o código usa `UNPACK_FLIP_Y_WEBGL`, para inverter a imagem ao carregar.)
- **Cada vértice recebe um (s, t).** A GPU interpola (s, t) para cada fragmento (1.5), multiplica por largura e altura da textura e lê o texel correspondente.

Formas de gerar as coordenadas (slides):

- **Planar:** projeta a textura de um plano, como um projetor de slides.
- **Cilíndrica** e **esférica:** enrola a textura ao redor do objeto (s vem do ângulo).
- **Explícita por vértice (mapa UV):** o artista "desdobra" o modelo numa imagem, como a planificação de um dado. Pode haver **costuras** (seams) nas bordas das partes.



6.2 Dando coordenadas de textura (receita)

1. **Divida a imagem em células.** Uma imagem com c colunas e r linhas de partes iguais tem bordas em s = 0, 1/c, 2/c, …, 1 e t = 0, 1/r, …, 1.
   A coluna i (contando da esquerda, a partir de 0) vai de s = i/c a (i+1)/c. A linha j **contando de baixo** vai de t = j/r a (j+1)/r. A linha de cima da imagem é a que chega em t = 1.
2. **Desenhe o objeto e dê nome aos vértices** (A, B, C…). A prova pede isso.
3. **Para cada face,** decida qual célula ela mostra e em que orientação. O canto da face que deve mostrar o canto inferior esquerdo da célula recebe (s_min, t_min), e assim por diante.
4. **Confira as faces vizinhas:** se a imagem é contínua (uma planificação), a aresta compartilhada deve ligar as mesmas bordas no desenho.
5. **Lembre que o vértice se repete:** um canto de cubo aparece em 3 faces, com até 3 coordenadas diferentes.

> **Exemplo 1: a imagem inteira numa face**
> Face com vértices A (embaixo, esquerda), B (embaixo, direita), C (em cima, direita), D (em cima, esquerda): A(0,0), B(1,0), C(1,1), D(0,1).
> Para repetir a imagem 2 vezes em cada direção: A(0,0), B(2,0), C(2,2), D(0,2), com wrap REPEAT. É o que o CubeTextured.html da aula faz.

Exemplo 2: planificação de dado (Prova 3)

Grade de 4 colunas × 3 linhas: s em 0 · 0,25 · 0,5 · 0,75 · 1 e t em 0 · 1/3 · 2/3 · 1. Com a face "6" na frente (célula da coluna 2, linha do meio):
A(0,5 ; 1/3), B(0,75 ; 1/3), C(0,75 ; 2/3), D(0,5 ; 2/3). A face de cima ("4") usa t de 2/3 a 1; a de baixo ("3"), t de 0 a 1/3; e a faixa do meio dá a volta no cubo. A resolução da Prova 3 tem o desenho e a tabela das 6 faces.

Exemplo 3: atlas em faixa (Prova 2)

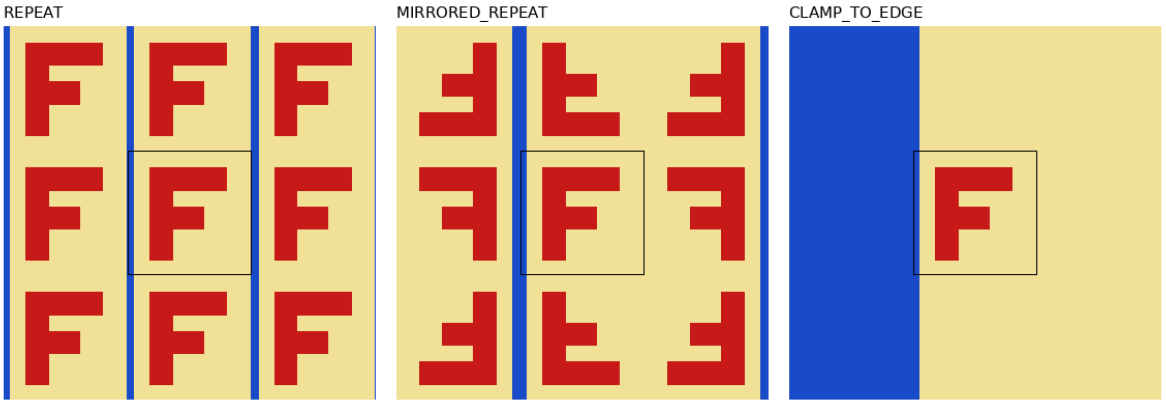
Quatro texturas lado a lado: s de 0 a 0,25 (paralelepípedo), 0,25 a 0,5 (concreto), 0,5 a 0,75 (pedras), 0,75 a 1 (tijolos). Cada parte da cena usa só a sua faixa de s. Para repetir **ao longo da rua**, varia-se t de 0 a N, o que funciona porque cada faixa ocupa a altura toda.

Atlas de textura: vantagem e desvantagem (Prova 3, questão 1)

Vantagem: uma única textura para tudo; um só carregamento e bind, menos trocas de estado, fácil de pintar.
Desvantagens: memória desperdiçada nas áreas vazias (50% na planificação do dado); vértices duplicados (24 em vez de 8 num cubo); **vazamento de cor nas bordas** (texture bleeding), porque a filtragem e o mipmap misturam texels da célula vizinha; não dá para usar REPEAT numa célula interna.

6.3 Repetição e modos de wrap

Coordenadas fora de [0, 1] são permitidas. O que acontece depende do modo de **wrap** (`TEXTURE_WRAP_S` e `TEXTURE_WRAP_T`):



A mesma textura (um "F" com uma faixa azul à esquerda) vista de s, t = −1 a 2. O quadrado preto marca [0,1].

- **REPEAT:** usa só a parte fracionária (1,3 vira 0,3). A textura se repete igual.
- **MIRRORED_REPEAT:** repete espelhando a cada inteiro. Entre 1 e 2 a textura aparece refletida, e entre −1 e 0 também. É o modo usado no código da aula.
- **CLAMP_TO_EDGE:** não repete; a última linha ou coluna de texels é esticada para fora.

Truque do atlas (Prova 2)

Num atlas em faixa, só a **primeira** e a **última** textura podem ser repetidas em s com MIRRORED_REPEAT: a rua com s de −0,25 a 0,25 (o espelho em s = 0 devolve o próprio paralelepípedo) e o muro com s de 0,75 a 1,25 (o espelho em s = 1 devolve os tijolos). Com REPEAT daria errado: −0,25 viraria 0,75 (tijolo). Uma textura do meio do atlas só se repete dividindo a geometria em mais polígonos.

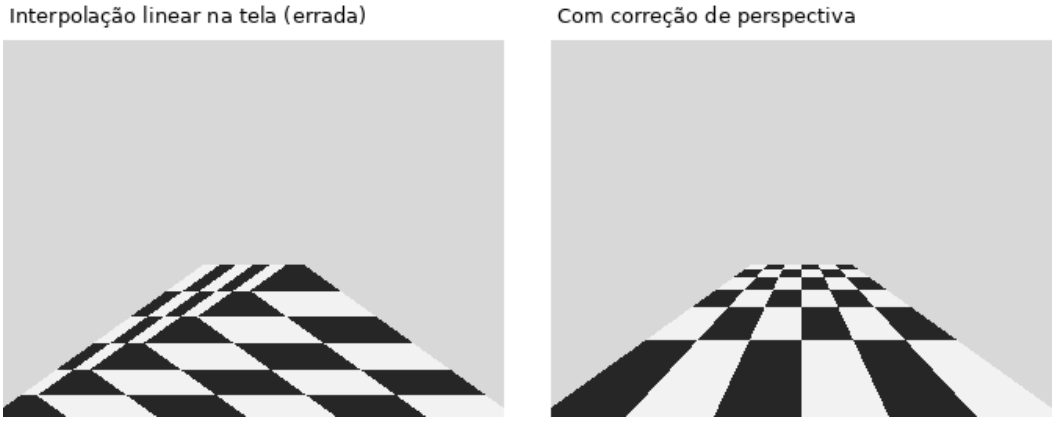
6.4 Diagnosticando mapeamentos errados

O que se vê	Causa provável
Listras: a cor muda numa direção e fica constante na outra, "arrastada" ao longo da face	Dois vértices ao longo dessa direção receberam a **mesma** coordenada (mesmo s ou mesmo t). A textura não varia ali, então uma única linha ou coluna de texels é esticada (Prova 1, questão 1).
Textura espelhada	As coordenadas de dois cantos estão trocadas (s de 1 a 0 em vez de 0 a 1), ou o wrap é MIRRORED.
Textura girada 90° ou 180°	As coordenadas foram atribuídas "deslocadas" de um ou dois vértices.
Imagem de cabeça para baixo	Diferença entre origem em cima e origem embaixo (falta o flip Y).
Textura repetida mais vezes que o desejado	Coordenadas maiores que 1 com REPEAT.
Linha quebrada na diagonal de um quadrilátero em perspectiva	Interpolação sem correção de perspectiva (6.5).
Cor de outra parte nas bordas	Vazamento no atlas (bleeding).

Na prova (Prova 1, questão 1)
Cubo com xadrez certo na frente e listras no topo e na lateral. Topo D(0,1), C(1,1), G(1,1), H(0,1): os vértices do fundo repetiram as coordenadas da frente, então t = 1 na face inteira e só a linha de cima da textura é esticada. Lateral B(1,0), C(1,1), G(1,1), F(1,0): s = 1 em tudo, então só a coluna da direita aparece.

6.5 Correção de perspectiva

Na projeção perspectiva, partes mais distantes de uma superfície ficam comprimidas na tela. Se a GPU interpolasse (s, t) **linearmente na tela**, trataria passos iguais em pixels como passos iguais na textura, e a textura entortaria. Como cada triângulo entorta de um jeito, aparece uma quebra na diagonal do quadrilátero.



Um chão com xadrez formado por 2 triângulos. À esquerda, interpolação linear na tela; à direita, com correção.

A correção: interpolar linearmente **s/w, t/w e 1/w** (que são lineares na tela) e, em cada fragmento, dividir: s = (s/w) / (1/w). As GPUs modernas fazem isso automaticamente.

> **Exemplo numérico**
> Uma aresta vai de um ponto a distância w = 1 (s = 0) até outro a w = 3 (s = 1). No pixel do **meio da tela**, a interpolação linear daria s = 0,5.
> O certo: s/w vale 0 e 1/3 nas pontas → meio = 1/6. 1/w vale 1 e 1/3 → meio = 2/3. s = (1/6)/(2/3) = **0,25**. A parte perto ocupa mais tela, então no meio da tela só se percorreu um quarto da textura.

6.6 Filtragem: nearest, bilinear, mipmap e anisotrópica

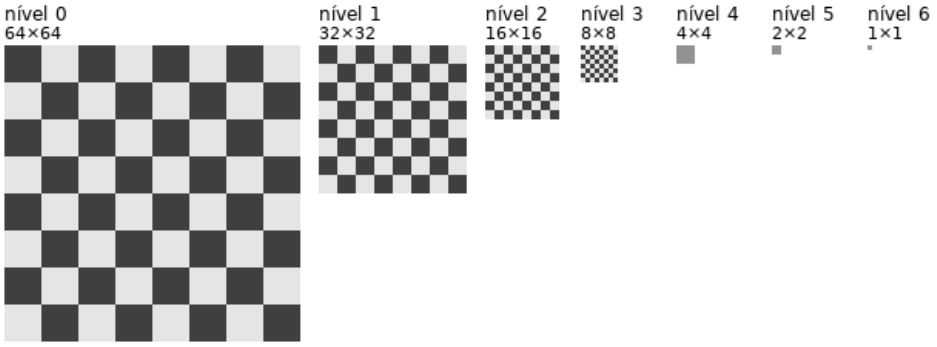
Quase nunca 1 pixel corresponde a exatamente 1 texel:

- **Magnificação** (textura ampliada, objeto perto): 1 texel cobre vários pixels. Filtro: `MAG_FILTER`.
- **Minificação** (textura reduzida, objeto longe ou inclinado): 1 pixel cobre vários texels. Filtro: `MIN_FILTER`.

Filtro	Como funciona	Resultado
NEAREST (amostragem pontual)	Pega o texel mais próximo	Ampliada: quadriculada (pixelada). Reduzida: **aliasing** (moiré, serrilhado, cintilação quando a câmera mexe).
LINEAR (bilinear)	Média ponderada dos 4 texels vizinhos	Ampliada: suave. Reduzida: ainda tem aliasing, porque 4 texels são poucos quando o pixel cobre 100.

Por que aparece aliasing: quando um pixel cobre muitos texels e só um deles é lido, os outros são ignorados (subamostragem). O texel escolhido depende da posição exata do pixel, por isso surgem padrões falsos (moiré) e pontos que piscam em movimento.

Mipmapping

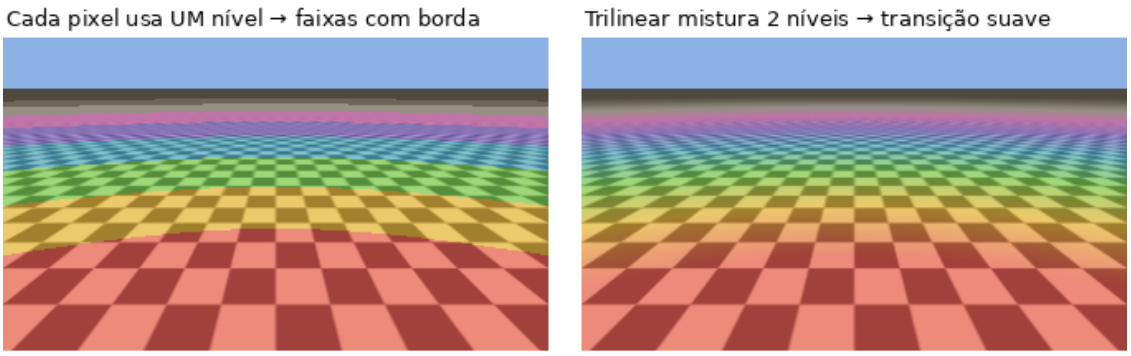


Pirâmide de mipmap: cada nível tem metade da resolução do anterior, até 1×1. Cada texel de um nível é a média de 4 do nível anterior.

- Pré-calcula versões cada vez menores e já filtradas da textura ("multum in parvo"). Na hora de desenhar, escolhe o nível em que **1 pixel ≈ 1 texel**.
- **Nível (LOD) = $\log_2(w)$**, em que w é quantos texels o pixel cobre. Exemplo: textura 256×256, pixel cobrindo 8 texels → LOD = 3 → nível 3 (32×32).
- **Custo de memória: só 33% a mais**, porque 1 + 1/4 + 1/16 + … = 4/3.
- Custo por pixel constante e baixo. Ainda ajuda o cache, porque os texels lidos ficam próximos.

Modo (MIN_FILTER)	Dentro do nível	Entre níveis
NEAREST_MIPMAP_NEAREST	texel mais próximo	um nível só
LINEAR_MIPMAP_NEAREST	bilinear (4 texels)	um nível só → **faixas visíveis**
NEAREST_MIPMAP_LINEAR	texel mais próximo	mistura 2 níveis
LINEAR_MIPMAP_LINEAR (trilinear)	bilinear	mistura 2 níveis → suave

A leitura do nome: **[filtro dentro do nível]_MIPMAP_[como trata os níveis]**.

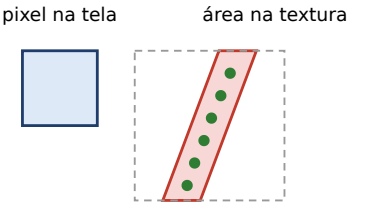


Níveis de mipmap pintados de cores diferentes. Usando um nível só por pixel, a troca de nível vira uma linha. A trilinear mistura os dois níveis vizinhos e a transição some.

Trilinear: `cor = lerp( bilinear(nível i), bilinear(nível i+1), LOD − i )`. São 8 texels por pixel e elimina a "linha" de transição entre níveis (Prova 1, questão 2b).

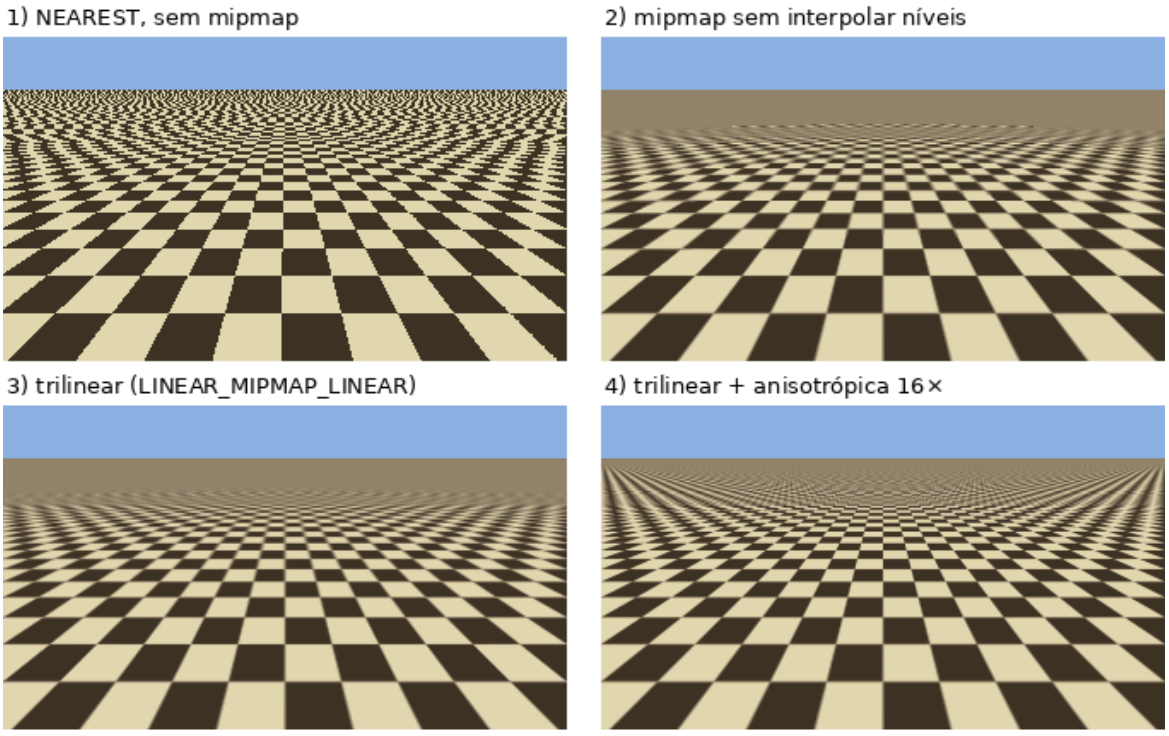
Filtragem anisotrópica

Numa superfície vista de lado (chão ao longe, uma rua), a área que um pixel cobre na textura é **comprida e estreita** (anisotrópica). O mipmap supõe que ela é um quadrado e escolhe o nível pelo lado **maior**, então borra também na direção em que não precisava. Resultado: **textura borrada à distância**.



tracejado: quadrado que o mipmap supõe
pontos: amostras da anisotrópica

Anisotrópica: aproxima a área por um retângulo (ou elipse) alongado, usa o **lado menor** para escolher o nível (mais nítido) e faz **várias amostras ao longo do lado maior** (2×, 4×, 8×, 16×). O ideal teórico é a EWA (média ponderada elíptica). No WebGL: extensão `EXT_texture_filter_anisotropic`, usada no código da aula.



Chão xadrez indo até o horizonte. 1) Sem mipmap: moiré e serrilhado ao longe. 2) Mipmap com um nível por pixel. 3) Trilinear: estável, mas borrado ao longe. 4) Trilinear + anisotrópica: nítido até longe.

Média de todos os texels (filtro de caixa) — Prova 3, questão 2

Calcular a média de **todos** os texels dentro da área do pixel (supersampling) resolve o aliasing de verdade, mas:

- **Desempenho:** o número de leituras cresce com a minificação (pode chegar a milhares por pixel), o custo varia de pixel para pixel e nada é reaproveitado entre quadros. É a "performance death spiral" dos slides.
- **Qualidade:** um retângulo alinhado aos eixos não acompanha a área real (inclinada, alongada); pesos iguais e entrada/saída brusca de texels causam cintilação; na ampliação (pixel menor que um texel) sobra 1 ou nenhum texel: imagem pixelada ou divisão por zero.
- **Alternativas:** mipmap (pré-filtragem, custo constante); trilinear; bilinear para ampliar; anisotrópica; **summed-area table** (tabela de somas acumuladas que dá a média de qualquer retângulo com 4 leituras).

Resumo de diagnóstico de filtragem

Sintoma	Causa	Solução
Moiré, serrilhado, cintilação ao longe	Minificação sem mipmap (subamostragem)	Mipmap (com trilinear)
Linha visível onde a nitidez muda	Mipmap usando um nível só por pixel	Trilinear (LINEAR_MIPMAP_LINEAR)
Textura borrada à distância, em ângulo rasante	Mipmap isotrópico escolhe nível pelo lado maior	Filtragem anisotrópica
Quadradinhos ao ver de perto	MAG_FILTER = NEAREST	MAG_FILTER = LINEAR

6.7 Bump mapping, environment mapping e transparência

- **Bump mapping:** simula rugosidade **sem mudar a geometria**. Uma segunda textura (mapa de altura, em tons de cinza) perturba a normal em cada ponto, e a iluminação reage como se houvesse relevo. A silhueta continua lisa, porque a forma não muda.

- **Normal mapping** (o mais comum hoje): a textura guarda a própria normal, em cores. Cada canal vai de 0 a 1 e é convertido com `n = 2·cor − 1`. Essa normal está no **espaço tangente** da superfície, e a matriz **TBN** (tangente, bitangente, normal da superfície) a leva para o mundo.
  Se T, B e N não coincidem com a geometria real, a luz fica errada (Prova 1, questão 4). A correção é calcular T, B e N de forma consistente para cada face e transformá-los pela matriz de modelo, como no cubebump.html.
- **Displacement mapping:** desloca de verdade os vértices (muda a silhueta). Precisa de uma malha bem subdividida.
- **Parallax / height map:** desloca as coordenadas de textura conforme o ângulo de visão, dando ilusão de profundidade (mais caro que normal mapping).
- **Environment mapping:** simula reflexos com imagens do ambiente. No **cube map** (6 imagens em volta do objeto), calcula-se o vetor de reflexão R; a face do cubo é escolhida pela **maior componente** de R (por exemplo, R = (0,3 ; 0,2 ; 0,8) → face +z), e as outras duas componentes, divididas por ela, dão a coordenada na face. Também serve para skybox e refração (lei de Snell).
- **Transparência (alpha blending):** mistura a cor do objeto da frente (a) com a do fundo (b): `C = αa·Ca + (1 − αa)·Cb`. Objetos transparentes são desenhados depois dos opacos, de trás para frente.

Parte 7 — Tabela de diagnóstico de imagens

Boa parte das questões mostra uma imagem com defeito e pergunta a causa e a solução. Use esta tabela como checklist.

O que se vê	Causa	Solução
Cores e geometria		
Degradê onde se queria cor sólida com borda reta	Cor por vértice é interpolada na rasterização	Subdividir a malha: a borda vira aresta; vértices da fronteira duplicados com cores diferentes (1.5)
Objeto não aparece	Fora do volume: além do far, atrás da câmera, fora das bordas	Ajustar câmera, far ou fov (4.5, 4.6)
Objeto cortado por um plano	Recortado pelo near ou pelo far	Aumentar o far, afastar a câmera ou ajustar o near
Objeto esticado ou achatado	Aspect da projeção diferente da proporção da janela	aspect = largura/altura
Objeto girou "em órbita" em vez de no lugar	Rotação em torno da origem	T(c)·R·T(−c) (3.4)
Iluminação		
Face cortada na diagonal: metade clara, metade escura	Os 2 triângulos da face têm normais diferentes (ordem dos vértices no produto vetorial)	Uma normal por face, ordem consistente, repetida nos 6 vértices (5.3)
Faces inteiras viradas para a luz escuras, as de trás claras	Vetor L com sinal invertido	L = luz − ponto
Iluminação estranha ao mover o objeto	Normal com w = 1 (pega a translação) ou sem normalizar	w = 0; normalizar; usar a matriz de normais
Aparência facetada, bandas na fronteira dos polígonos	Sombreamento constante + efeito de bandas de Mach	Gouraud ou Phong, normais por vértice (média)
Brilho especular sumiu ou ficou deformado	Gouraud: a luz só é calculada nos vértices	Phong (por fragmento) ou malha mais fina
Partes sem luz direta totalmente pretas	Falta a componente ambiente	Somar Ia·Ka
Rugosidade do normal map errada, luz desalinhada	Matriz TBN não coincide com a geometria	Calcular tangente, bitangente e normal corretas e transformá-las com o modelo (6.7)
Texturas		
Listras esticadas numa face	Dois vértices com a mesma coordenada de textura ao longo dessa direção	Variar s ou t entre esses vértices (6.4)
Textura entortada, quebrada na diagonal	Interpolação sem correção de perspectiva	Interpolar s/w, t/w, 1/w (6.5)
Moiré, cintilação ao longe	Minificação sem mipmap (aliasing)	Mipmap
Linha onde a nitidez muda bruscamente	Mipmap sem interpolar níveis	Trilinear
Textura borrada à distância	Mipmap isotrópico em ângulo rasante	Anisotrópica
Quadradinhos de perto	Magnificação com NEAREST	MAG_FILTER = LINEAR
Cor da parte vizinha nas bordas	Vazamento (bleeding) no atlas	Margem (padding) entre as células

Parte 8 — A planilha, passo a passo

8.1 Para que serve

A questão de MVP tem dezenas de multiplicações 4×4 e um erro numa delas estraga tudo. A planilha faz todas as contas e mostra cada passo intermediário, que a prova pede. Ela segue exatamente a receita da 4.8. Há três arquivos:

Arquivo	Conteúdo
Prova2_Q3_MVP.xlsx	Um triângulo, rotação central em Z + escala, câmera com **FOV vertical**. É o modelo mais geral; use este para praticar.
Prova3_Q3_MVP.xlsx	O mesmo layout, mas a entrada de FOV é **horizontal** (a célula I43 converte para vertical).
Q6_MVP_Prova.xlsx	Dois triângulos (T e U) da Prova 1, com duas abas: a câmera original e a câmera modificada.

Abra no LibreOffice Calc ou no Excel. Não precisa programar nada.

8.2 Legenda de cores

- amarelo com texto azul: **entradas** que você pode (e deve) mudar: dígitos, ângulos, escala, câmera.
- Texto preto: **fórmulas**. Não digite em cima delas (a exceção é o target, explicada abaixo).
- Blocos 4×4 com borda: **matrizes**, cada uma com o nome em negrito logo acima.
- Cabeçalhos em azul-claro: títulos de colunas e blocos.

8.3 Mapa da planilha (Prova2_Q3_MVP.xlsx; a da Prova 3 é igual)

Linhas	Colunas B a E	Colunas H a K	Colunas N a Q	Teoria
4–6	Entradas: X (C5) e Y (C6)	—	—	4.8 passo 1
8–12	Triângulo original A, B, C (C9:E11); centro c (C12:E12)	Parâmetros: ângulo (I9) e escala (I10)	—	3.4
15–19	T1 = T(−c)	R = Rz(ângulo)	T2 = T(+c)	3.1, 3.4
21–25	aux = R · T1	Rc = T2 · R · T1	S = escala	3.3
27–31	**M = S · T2 · R · T1**	Triângulo no mundo A', B', C' (I28:K30); centro final (I31:K31)	—	3.5
34–39	Câmera: eye (C36:E36), target (C37:E37), up (C38:E38)	Largura, altura, FOV, near, far (I35:I39)	—	4.2, 4.3
40–45	at − eye, v, v × up, n, u (C41:E45)	aspect (I40), f (I41), d (I42) [Prova 3: fovy em I43]	—	4.2, 4.3
47–51	V (view)	P (projeção)	PV = P · V	4.2, 4.3
53–58	**MVP = P · V · M** (B55:E58)	—	—	4.1
60–64	Posições finais de A, B, C: clip (C–F), NDC (H–J), pixel na tela (L–M), situação (N)			4.4, 4.5
66	Resumo: quantos vértices dentro e a conclusão			4.5

8.4 Usando, passo a passo

1. **Digite seus dígitos:** X (último) em **C5** e Y (penúltimo) em **C6**. Tudo recalcula sozinho.
2. **Confira os vértices** em C9:E11: devem bater com o enunciado já com seus dígitos.
3. **Confira o centro** c em C12:E12 (média dos três vértices).

4. **Ângulo e escala** em I9 e I10. O ângulo vai em **graus** (a planilha converte). Negativo = sentido horário.
5. **Copie as matrizes intermediárias** para a folha de resposta, com os nomes: T1, R, T2, Rc, S e M.
6. **Triângulo no mundo:** I28:K30 mostra onde cada vértice foi parar depois de M.
7. **Câmera:** confira eye, up, largura, altura, FOV, near e far conforme o enunciado. O target (C37:E37) vem preenchido com o **centro do triângulo final**; se o professor quiser o centro original, digite c por cima.
8. **V e P** aparecem nos blocos das linhas 48–51; copie-os.
9. **Matriz final:** MVP em B55:E58. É a resposta de "forneça a matriz final".
10. **Posição final** (linhas 62–64): clip (x, y, z, w), NDC (dividido por w) e pixel. A coluna N diz se cada vértice está DENTRO, ALÉM DO FAR, FORA etc.
11. **Conclusão** (linha 66): totalmente visível, parcialmente ou nenhum vértice dentro. Escreva na prova o **motivo** com a tabela da 4.5 (por exemplo: "A e C têm z_ndc > 1, além do far").

8.5 Entendendo o que cada bloco faz

- **T1 = T(−c):** identidade com −cx, −cy, −cz na última coluna. Leva o centro para a origem.
- **R:** a matriz de rotação da 3.1, com COS e SEN do ângulo.
- **T2 = T(+c):** igual à T1, com sinal trocado. Devolve o centro.
- **aux e Rc:** produtos parciais. Rc é "girar em torno do centro".
- **S e M:** a escala e a matriz de modelo completa.
- **Vetores da câmera:** "at − eye" é a direção sem normalizar; v é ela normalizada; "v × up" e n são a direita; u é o cima.
- **V:** linhas n, u e −v; última coluna −n·eye, −u·eye, +v·eye.
- **P:** f/aspect, f, −(far+near)/d, −2·far·near/d e o −1 na última linha.
- **MVP:** o produto P·V·M. Cada linha da tabela final é MVP·(x, y, z, 1) seguido da divisão por w.

8.6 Adaptando a planilha a outra questão

Se a questão mudar…	Faça
Os vértices	Digite os novos valores (ou fórmulas com C5 e C6) em C9:E11.
A rotação for em torno da **origem**	Digite 0 em C12, D12 e E12. Com c = 0, T1 e T2 viram identidade e sobra só R.
O eixo de rotação for X ou Y	Reescreva o bloco R (H16:K19) com o padrão da 3.1. Rx: 1 no canto, `=COS(RADIANS($I$9))` e `=-SIN(RADIANS($I$9))` nas linhas 2 e 3. Ry: COS nos cantos de x e z, SIN em cima à direita, −SIN embaixo à esquerda.
Não houver escala	Escala = 1 em I10.
Houver um deslocamento extra, ou outra ordem	Crie um bloco 4×4 novo com a matriz e refaça o produto com MMULT (abaixo).
O FOV for horizontal	Use a planilha da Prova 3 (I37 é horizontal), ou calcule fovy = 2·atan(tan(fovx/2)/aspect) e digite na da Prova 2.
O target for outro ponto	Digite por cima de C37:E37.
Dois triângulos	Use a Q6_MVP_Prova.xlsx.

Multiplicar matrizes no LibreOffice ou no Excel (MMULT)

1. Selecione um bloco vazio de **4 linhas × 4 colunas**.
2. Digite `=MMULT(H16:K19; B16:E19)` : primeira matriz, ponto e vírgula, segunda. A ordem importa: R·T1 é `MMULT(R; T1)` .
3. Confirme com **Ctrl + Shift + Enter** (fórmula matricial). Só Enter preenche uma célula só.
4. Para três matrizes: `=MMULT(A; MMULT(B; C))` . Para matriz × ponto, a segunda faixa é uma coluna de 4 células (x, y, z, 1).

No Excel em português os nomes são MATRIZ.MULT, COS, SEN e RADIANOS.

8.7 Conferências rápidas (para pegar erro)

- Numa rotação central, o **centro não se move**: Rc·c = c.
- v, n e u devem ter comprimento 1.
- V · eye = (0, 0, 0, 1): a câmera vai para a origem.
- O **target**, depois de MVP e divisão por w, cai em **x_ndc = 0, y_ndc = 0** (centro da tela). Se não cair, há erro na V.
- O w_clip de um ponto é a sua distância à frente da câmera (positivo = na frente).
- Um ponto exatamente no near dá z_ndc = −1; no far, z_ndc = +1.

8.8 A planilha do professor (ExercícioViewProjectionCorreto2024.xlsx)

Ela faz a mesma conta com outros nomes. Câmera com eye (0, −5, 5), target (5, 3, 2) e up (0, 0, 1); pontos (4,2,2), (6,2,2), (6,4,2) e (4,4,2) (um quadrado); janela 800×600, fov 45°, near 1, far 12.

Onde	O quê
B2:B5, B7:B10, B12:B15	Target, up e eye (com 1 ou 0 na 4ª linha)
H2:H4 (w), F2:F4 (u), G2:G4 (v)	w = frente normalizada; u = w × up (direita); v = u × w (cima)
L2:O5	Matriz de visualização: linhas u, v, −w normalizadas; última coluna − (linha · eye)
F7:F10 → H7:H9	Um ponto e o mesmo ponto no espaço da câmera (MMULT)
F12:L15	Os pontos já no espaço da câmera (valores digitados)
L18:L21, K23	r (aspect), alfa (fov), near, far e d = far − near
J26:M29	Matriz de projeção, igual à da 4.3
F30:L33	Pontos no espaço de recorte (P × ponto da câmera)
F35:L38	NDC (divididos por w)

Armadilha na planilha do professor

A célula **J24**, tan(fov/2), está **digitada** como 0,4142 (tan 22,5°, para fov = 45°), não é fórmula. Se mudar o fov em L19, atualize J24 à mão. Os pontos no espaço da câmera (F12:L15) também estão digitados; se mudar a câmera, recalcule-os com a matriz L2:O5.

Planilha de outro estudante (AutoCGSheet): cuidados

O texto dela diz `VP = mult(V, P)` e `MVP = mult(M, VP)` , na ordem errada (o certo é P·V e P·V·M; os cálculos dela estão certos). A média dos pontos divide sempre por 3, mesmo com 2 pontos. A fórmula de pixel do texto (y+1)/altura/2 deveria ser (y+1)·altura/2.

Parte 9 — As três provas, questão por questão

Prova 1 (2025)

Questão	Resposta curta	Seção
1. Cubo com xadrez certo numa face e listras nas outras: nomear vértices e dar as coordenadas usadas	Frente A(0,0), B(1,0), C(1,1), D(0,1). Topo: G(1,1), H(0,1) repetem C e D, então t = 1 constante e só a linha de cima é esticada. Lateral: F(1,0), G(1,1), s = 1 constante.	6.2, 6.4
2a. Por que interpolar linearmente na tela distorce a textura	A projeção divide por z (não é linear); passos iguais na tela ≠ passos iguais na superfície. Corrige-se interpolando s/w, t/w, 1/w.	6.5
2b. Artefato com mipmap (linha de transição) e solução	Um nível de mipmap por pixel → troca brusca de resolução. Solução: trilinear (bilinear em 2 níveis + lerp pelo LOD). Complemento: anisotrópica para o borrão.	6.6
3. Cubo iluminado com faces cortadas na diagonal; avaliar a resposta do "Gepeto"	Discordar das duas explicações. Sinal de L afetaria faces inteiras; normal por face é correta. Erro: normais diferentes nos 2 triângulos da face (ordem no produto vetorial).	5.2, 5.3
4. Normal map com a normal base errada (não cai)	TBN inconsistente com a geometria: calcular T, B, N corretos por face e transformá-los com a matriz de modelo.	6.7
5. Quadrado com degradê verde/vermelho	Interpolação de cor por vértice. Subdividir a malha, com vértices da fronteira duplicados.	1.5
6. MVP de dois triângulos; visíveis? mudar a câmera	M_T = S(0,5)·Rz(45°); M_U = T(2,0,0)·S(2)·T(c)·Rx(−60°)·T(−c). Nenhum aparece: T além do far, U atrás e abaixo da câmera. Câmera em (−11, −18, −17) com far 40: U (grande) cobre T.	3, 4, 8

Prova 2

Questão	Resposta curta	Seção
1. Atlas (4 texturas) aplicado a rua, meio-fio, calçada e muro	Faixas de s: 0–0,25, 0,25–0,5, 0,5–0,75, 0,75–1. t de 0 a N ao longo da rua. Rua e muro repetem 2× com MIRRORED_REPEAT (s −0,25 a 0,25 e 0,75 a 1,25).	6.2, 6.3
2. Textura borrada à distância	Área do pixel alongada (anisotrópica); o mipmap escolhe o nível pelo lado maior. Solução: filtragem anisotrópica.	6.6
(Iluminação) Dois planos, mesmo modelo, resultados diferentes	Gouraud (luz nos vértices, perde o brilho) × Phong (luz por pixel). Vantagens e desvantagens de cada um.	5.4
3. Matriz final, intermediárias e posição final	M = S(2)·T(c)·Rz(60°)·T(−c); V, P, MVP; A e C sempre além do far = 25. Parcialmente visível em 89 de 100 matrículas.	3, 4, 8

Prova 3 (2024)

Questão	Resposta curta	Seção
1. Planificação do dado num cubo; vantagem e desvantagem	Grade 4×3; nomear A–H; (s, t) de cada face. Vantagem: uma textura só. Desvantagens: área vazia, vértices duplicados, vazamento nas bordas.	6.2
2. Algoritmo do João (média dos texels da área)	(a) resolve o aliasing de amostrar 1 texel; (b) custo variável e alto, retângulo alinhado aos eixos, problemas na ampliação (N = 0 ou 1); (c) mipmap, trilinear, anisotrópica, SAT.	6.6
3. Matriz final, posição e visibilidade	FOV horizontal 120° → fovy 81,79°. Nunca visível por inteiro: far = 10 curto, triângulo grande e visto de lado. Câmera (3, 0, 30) e far 40 resolve.	3, 4, 8

Padrão das provas

Sempre há: (1) uma questão de **coordenadas de textura**; (2) uma de **filtragem** (mipmap, trilinear, anisotrópica); (3) uma de **iluminação** (normais ou Gouraud × Phong); (4) a grande de **MVP** (vale metade da nota), sempre com rotação em torno do centro, escala, câmera, projeção e "é visível?".

Parte 10 — Glossário e folha de fórmulas

Glossário

Aliasing	Serrilhado, moiré e cintilação por amostrar pouco (um texel por pixel).
Anisotrópica	Filtragem que respeita a forma alongada da área do pixel na textura.
Aspect ratio	Proporção largura/altura.
Atlas de textura	Várias texturas numa imagem só.
Baricêntricas	Pesos dos 3 vértices usados para interpolar dentro do triângulo.
Clip space (espaço de recorte)	Resultado de P·V·M, antes da divisão por w.
Clipping (recorte)	Cortar o que está fora do volume visível.
Culling	Descartar o que não pode aparecer (faces de trás, objetos fora do campo de visão).
Far / near	Planos mais distante e mais próximo que aparecem.
Fragmento	Um pixel candidato gerado pela rasterização de um triângulo.
Framebuffer	Memória com a imagem: buffer de cor + Z-buffer.
Frustum	Tronco de pirâmide que a câmera enxerga.
FOV (fovy)	Campo de visão (vertical).
LOD	Nível de detalhe; no mipmap, $\log_2$ do tamanho da área em texels.
Magnificação / minificação	Textura ampliada / reduzida na tela.
Mipmap	Pirâmide de versões menores da textura (+33% de memória).
NDC	Coordenadas normalizadas do dispositivo: cubo de -1 a 1.
Normal	Vetor unitário perpendicular à superfície.
Ocluso	Dentro do volume, mas atrás de outro objeto (Z-buffer).
Pipeline	Sequência de etapas de vértices até pixels.
Rasterização	Descobrir quais pixels um triângulo cobre.
Shader (vertex / fragment)	Programa na GPU, por vértice ou por pixel.
Shading (sombreamento)	Onde a luz é calculada: constante, Gouraud ou Phong.
Texel	Pixel da textura.
Trilinear	Bilinear em 2 níveis de mipmap + interpolação entre eles.
Uniform	Valor igual para todos os vértices (matrizes, luz).
Viewport	Área da janela em pixels onde a imagem é desenhada.
Wrap (REPEAT, MIRRORED, CLAMP)	O que fazer com coordenadas de textura fora de [0,1].
Z-buffer	Profundidade de cada pixel, para decidir quem fica na frente.

Folha de fórmulas

Vetores

```
|v| = √(x² + y² + z²)      v̂ = v/|v|
a·b = ax·bx + ay·by + az·bz = |a||b|cos θ
a×b = (ay·bz − az·by, az·bx − ax·bz, ax·by − ay·bx)
centróide = (A + B + C)/3
```

Transformações

```
T(dx,dy,dz): última coluna (dx,dy,dz,1)
S(s): diagonal (s,s,s,1)
Rz: [c −s; s c] no canto (x,y)
Rx: [c −s; s c] em (y,z)
Ry: [c s; −s c] em (x,z)
Em torno de c: T(c)·R·T(−c)
Ordem: M = T₃·T₂·T₁ (o último aplicado à esquerda)
Ponto w = 1, vetor (normal) w = 0
```

Iluminação

```
I = Ia·Ka + Kd·I·max(N·L,0) + Ks·I·max(R·V,0)ⁿ
R = 2(N·L)N − L      H = normalize(L + V)
L = luz − ponto  (do ponto para a luz)
```

Câmera

```
v = normalize(target − eye)
n = normalize(v × up)
u = n × v
V = [ n | −n·eye ]
    [ u | −u·eye ]
    [−v |  v·eye ]
    [ 0 0 0 | 1  ]
```

Projeção

```
f = 1/tan(fovy/2)     d = far − near
P = [f/asp 0  0              0          ]
    [0     f  0              0          ]
    [0     0  −(far+near)/d  −2·far·near/d]
    [0     0  −1             0          ]
tan(fovx/2) = aspect · tan(fovy/2)
```

Do vértice ao pixel

```
MVP = P·V·M        clip = MVP·(x,y,z,1)
NDC = (x/w, y/w, z/w)
dentro: w > 0 e −w ≤ x,y,z ≤ w
x_px = (x_ndc+1)·W/2   y_px = (y_ndc+1)·H/2
```

Texturas

```
célula (col i, linha j de baixo), grade c×r:
s ∈ [i/c, (i+1)/c]   t ∈ [j/r, (j+1)/r]
LOD = log₂(texels por pixel)
memória do mipmap = 4/3 do original
trilinear = lerp(bil(i), bil(i+1), LOD − i)
alpha: C = α·Ca + (1 − α)·Cb
```